JUSTIFICACIÓN

Daniel Perez

Reto 1: Construcción Multietapa y Huella Estricta

Objetivo y Enfoque: Se diseñó un `Dockerfile` dividido en dos etapas claras (multietapa) con el fin de desacoplar el entorno de compilación del entorno mínimo de ejecución en producción. **Implementación:**

- **Etapa de Compilación (`builder`):** Se utilizó la imagen `maven:3.9.6-eclipse-temurin-21-alpine`. Se aisló primero la copia del archivo `pom.xml` y se ejecutó `mvn dependency:go-offline -B` antes de transferir el código fuente (`src/`). Esto permite aprovechar la caché de capas de Docker en compilaciones futuras cuando las dependencias no cambian.
- **Etapa de Ejecución (`runtime`):** Se utilizó `eclipse-temurin:21-jre-alpine` para garantizar un entorno ligero y sin herramientas innecesarias de desarrollo.
- **Seguridad y Privilegios:** Se creó el grupo `sensorgroup` y el usuario del sistema `sensor_user` con UID 10001 (`adduser -S sensor_user -G sensorgroup -u 10001`). Se asignó la titularidad del directorio `/app` a este usuario y se cambió a `USER sensor_user` antes del `ENTRYPOINT`, asegurando que la aplicación jamás se ejecute como `root`.
- **Trazabilidad y JVM:** Se inyectó el identificador del estudiante mediante `ARG STUDENT_ID` en tiempo de construcción y se asignó a la variable de entorno `ENV OPERATOR_ID=${STUDENT_ID}`. Además, se configuró `JAVA_OPTS` con `-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0` para que la máquina virtual

respete las restricciones impuestas por el contenedor.

Reto 2: Auditoría Forense y Comprobación de Vulnerabilidades

Objetivo y Enfoque: Validar y certificar en caliente las medidas de seguridad, inmutabilidad y aislamiento de privilegios aplicadas en la imagen. **Implementación y Evidencias:**

- **Desacoplamiento de Capas:** Con `docker history`, se auditó el tamaño y los comandos de creación de cada capa, verificando que la etapa final no arrastre código fuente ni el peso del SDK de Maven.
- **Aislamiento de Procesos:** Con `docker top eco_air_active -o pid,user,comm`, se comprobó que el comando `java` corre bajo el usuario no privilegiado (`sensor_user` / UID 10001) y no bajo `root`.
- **Inmutabilidad de Runtime:** Al intentar crear un archivo en el sistema operativo mediante `docker exec eco_air_active touch /etc/crontab_fake`, el sistema rechazó la operación por falta de permisos (Permission denied), confirmando que un atacante no puede alterar configuraciones del sistema.
- **Auditoría de Artefactos de Desarrollo:** Se verificó que ni `mvn` (`which mvn`) ni archivos de código fuente Java (`find / -name "*.java"`) existen dentro del contenedor en ejecución, reduciendo la superficie de ataque al mínimo.

Reto 3: Calibración de cgroups y Comportamiento OOM

Objetivo y Enfoque: Someter el contenedor a estrés de memoria para verificar las políticas de contención del kernel y calibrar los límites operativos. **Implementación y Resultados:**

- **Inducción de Colapso (OOMKilled):** Se ejecutó el contenedor

limitando la memoria a 48 MB (`docker run -d --name eco_air_crash -m 48m eco-air-backend:1.0.0`). Al no ser suficiente para el inicio de Spring Boot, el kernel invocó al OOM Killer, verificando el fallo controlado mediante `docker inspect` (`OOMKilled: true`).

- **Calibración y Estabilidad:** Se ajustó la cuota de memoria para encontrar el umbral donde el contenedor arranca en menos de 15 segundos y mantiene un consumo estable reportado en `docker stats --no-stream`.